

Módulo 4:

Agent Skills

CLAUDE

COZE

Recap:

1- LLM Stateless + Claude Code

2- Setup + CLAUDE.md

3- Context Engineering

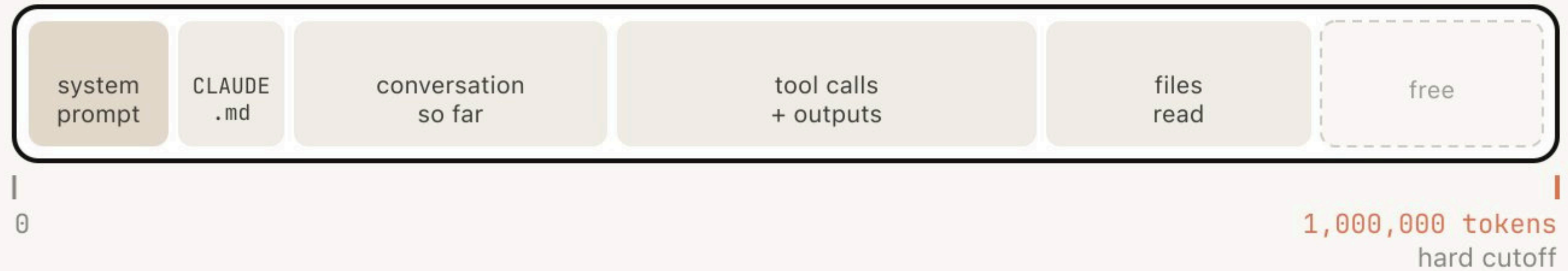
LLM

Knowledge

**Procedural
Knowledge**

The context window

Everything the model can see when it generates the next response



SKILL.md

- **YAML**
- **Markdown**

Frontmatter: YAML

examples >  SKILL.md

```
1  ---
2  name: standup
3  description: Generates a daily standup note. Use when the user says
   "standup", "daily", "scrum update", or asks to prepare their standup
   for the day.
4  ---
5
```

Body: Markdown

```
5
6  ## Instructions
7
8  Ask the user these three questions one by one:
9
10 1. What did you work on yesterday?
11 2. What are you working on today?
12 3. Any blockers?
13
14 Then output the standup in this format:
15
16 **Yesterday:** <answer>
17 **Today:** <answer>
18 **Blockers:** <answer or "None">
19
```

A simple SKILL.md file

pdf/SKILL.md

YAML Frontmatter

```
----
name: pdf
description: Comprehensive PDF toolkit for extracting text and tables,
merging/splitting documents, and filling-out forms.
----
```

Markdown

Overview

This guide covers essential PDF processing operations using Python libraries and command-line tools. For advanced features, JavaScript libraries, and detailed examples, see `./reference.md`. If you need to fill out a PDF form, read `./forms.md` and follow its instructions.

Quick Start

```
```python
from pypdf import PdfReader, PdfWriter
```

#### # Read a PDF

```
reader = PdfReader("document.pdf")
print(f"Pages: {len(reader.pages)}")
```

```
...
```



# Basic Frontmatter

- name: skill name (max 64 chars)
- description: What it does and when to use it.  
This is what Claude reads to decide whether to invoke it.

# Advance Frontmatter

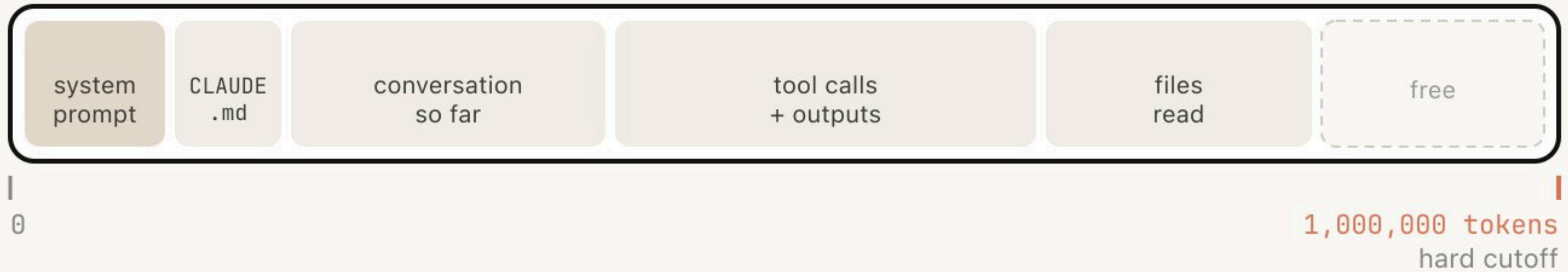
- when\_to\_use
- argument-hint
- arguments
- disable-model-invocation
- user-invocable
- allowed-tools
- model
- effort
- context
- agent
- hooks
- paths
- shell

examples > skill\_progressive\_disclosure > skill-with-scripts > SKILL.md

```
1 ---
2 name: git-msg
3 description: Generates a conventional git commit message from staged changes. Use when the
4 user asks to write a commit message or summarize what they are about to commit.
5 argument-hint: "feat|fix|chore"
6 arguments: type
7 disable-model-invocation: true
8 allowed-tools: Bash
9 model: claude-haiku-4-5-20251001
10 effort: low
11 ---
12 Run this to see what is staged:
13
14 ```!
15 git diff --cached --stat
16 git diff --cached
17 ```
18
19 Write a single commit message in conventional commits format using the type $type.
20
21 Format:
22 ```
23 $type: <short description in imperative mood>
24 ```
25
26 Rules:
27 - One line only
28 - Under 72 characters
29 - No period at the end
30 - Describe what the change does, not how
31
```

# The context window

Everything the model can see when it generates the next response



# Progressive Disclosure

## Bundling additional content

pdf/SKILL.md

### YAML Frontmatter

```

name: pdf
description: Comprehensive PDF toolkit for extracting text and
tables, merging/splitting documents, and filling-out forms.

```

### Markdown

#### ## Overview

This guide covers essential PDF processing operations using Python libraries and command-line tools. For advanced features, JavaScript libraries, and detailed examples, see `./reference.md`. If you need to fill out a PDF form, read `./forms.md` and follow its instructions.

#### ## Quick Start

```
```python
from pypdf import PdfReader, PdfWriter
```

```
# Read a PDF
reader = PdfReader("document.pdf")
print(f"Pages: {len(reader.pages)}")
```

```
# Extract text
text = ""
for page in reader.pages:
    text += page.extract_text()
```

pdf/reference.md

PDF Processing Advanced Reference

This document contains advanced PDF processing features, detailed examples, and additional libraries not covered in the main skill instructions.

pypdfium2 Library (Apache/BSD License)

Overview

pypdfium2 is a Python binding for PDFium (Chromium's PDF library). It's excellent for fast PDF rendering, image generation, and serves as ...

pdf/forms.md

If you need to fill out a PDF form, first check to see if the PDF has fillable form fields. Run this script from this file's directory:

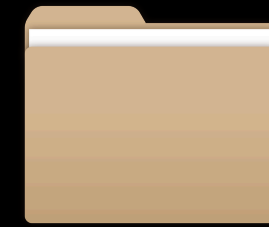
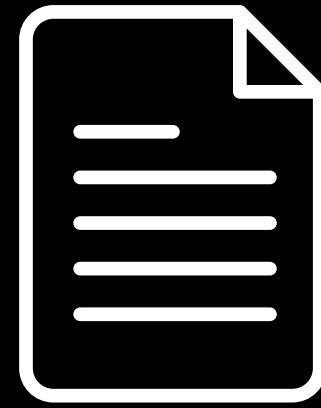
```
`python scripts/check_fillable_fields <file.pdf>`,
and depending on the result go to either the "Fillable
fields" or "Non-fillable fields" and follow those
instructions.
```

Fillable fields

If the PDF has fillable form fields:

```
- Run this script from this file's directory:
`python scripts/extract_form_field_info.py <input.pdf>
<fields.json>`.
...
```


SKILL.md



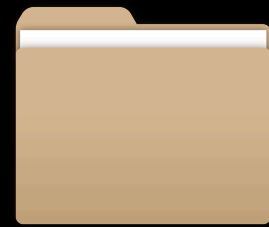
scripts

script.py



references

reference.md



assets

assets.txt

Progressive Disclosure

```
5
6  ## Step 1 – Get file stats (script)
7
8  Run the stats script on the file:
9
10  ```!
11  python3 ${CLAUDE_SKILL_DIR}/scripts/stats.py $ARGUMENTS
12  ```
13
14  ## Step 2 – Check style (references)
15
16  Read references/style-rules.md and identify up to 3 rules the file violates.
17
18  ## Step 3 – Write the review (assets)
19
20  Use the template in assets/report-template.md to format your output.
21  Fill every section. Be direct and specific – name the line or sentence that has the issue.
22  |
```

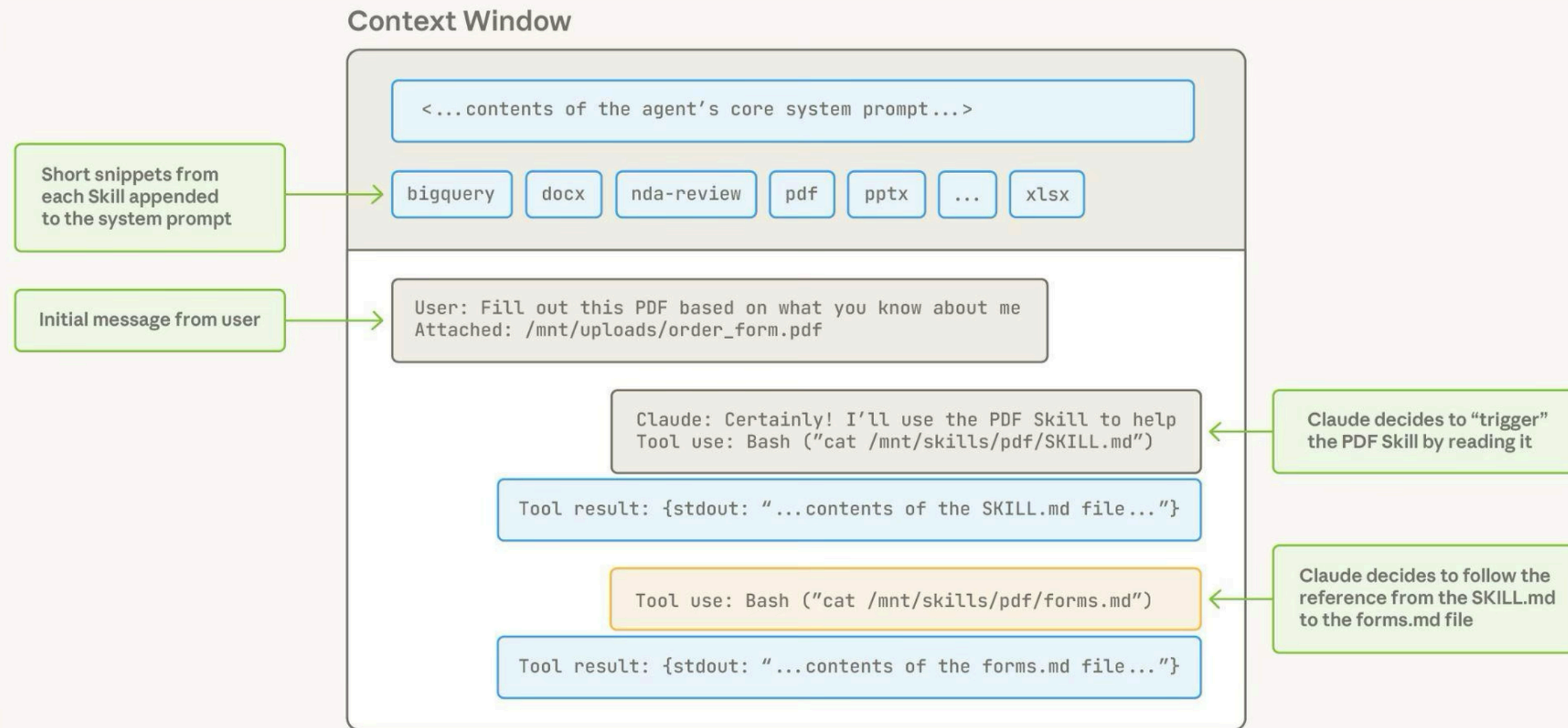
scripts: output

references

assets

Agent Skills

Skills and the Context Window



Gracias!

Linkedin: Daniel Ávila Arias

Twitter: @dani_avila7

Instagram: danielsan_avila

Github: davila7

Medium: @dan.avila7

Linkedin: Camila Banares

Carrasco

Instagram: @camidevai

Github: camidevai

Facebook: camicaturadigital